STORELEDGER

Building a Database-Driven Billing & Inventory System

A portfolio case study on turning DBMS concepts into a working Flask + MySQL application.

Python	Flask	MySQL	JavaScript	CSS	Git

Tushar Thakur

MCA Data Science · Chandigarh University

Project Type DBMS Mini Project	**Interface** Web Application	**Database** MySQL	**Launch** One-click BAT
Auth Session + Hashing	**Payments** Cash / Card / UPI / Wallet	**Reports** Sales + Category	**Theme** Light / Dark

PROJECT STORY

Why I Built StoreLedger

I wanted this project to be more than a collection of CRUD pages. The goal was to build a compact store-management system where the database actively participates in the business logic. StoreLedger therefore combines a practical billing workflow with inventory, customers, suppliers, wallets, rewards, reports, expiry tracking, authentication, and audit logging.

The result is a project that is easy to demonstrate in a classroom, but still shows how frontend actions, Flask routes, SQL queries, stored programming, and transactions work together in a real application.

CONTEXT

The Problem

Typical small-store pain points

Separate records for stock, bills, customers and suppliers make it difficult to keep data consistent and trace changes.

What StoreLedger does

Brings core store operations into one database-backed workflow and validates important rules at both application and database level.

KEY CAPABILITIES

What the Application Can Do

Billing & invoices

Create bills, calculate tax and discounts, validate stock, choose payment methods, and view/download invoice text.

Inventory & expiry

Track stock, minimum levels, out-of-stock items, restocking, low-stock alerts, and product expiry status.

Customers & wallet

Maintain customer records, wallet balances, wallet transactions, reward points, and purchase history.

Products, categories & suppliers

Manage products, categories, tax rates, prices, stock, expiry dates, and supplier relationships.

Reports & dashboard

See daily revenue, order counts, top-selling products, payment distribution, category reports, and alerts.

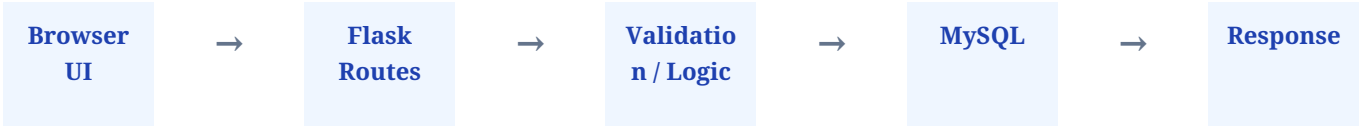
Security & audit

Protect routes with sessions and password hashing, while database triggers record selected product changes.

ARCHITECTURE

How StoreLedger Is Structured

The application follows a straightforward three-layer flow: the browser provides the interface, Flask handles requests and business orchestration, and MySQL stores data while also executing database-side logic.



Layer	Main Technology	Responsibility
Presentation	HTML + CSS + JavaScript	Forms, tables, cart interactions,

Layer	Main Technology	Responsibility
		responsive UI and dark mode.
Application	Python + Flask	Routes, validation, sessions, calculations, transaction orchestration and rendering.
Database	MySQL	Relational data, constraints, procedures, functions, triggers, views and transaction safety.

EXAMPLE WORKFLOW

Data Flow: Generating a Bill

Select items → **Validate cart** → **Lock products** → **Create order** → **Update stock** → **Commit** → **Show invoice**

The billing flow uses server-side validation and database transactions. Product rows are locked with SELECT ... FOR UPDATE before stock is changed, which helps prevent overselling when multiple operations target the same product.

CORE RELATIONSHIPS

Database Model

Parent	Relationship	Child / Usage
Category	1 → many	Products
Supplier	1 → many	Products
Customer	1 → many	Orders / Wallet Transactions
Order	1 → many	Order Items

DATABASE ENGINEERING

The DBMS Work Behind the UI

A major goal of StoreLedger was to demonstrate that the database is not just a passive storage layer. Several important rules are implemented directly in MySQL.

Object	Examples	Purpose
Functions	calculate_discount,	Reusable calculations executed

	calculate_reward_points	inside MySQL.
Procedures	process_order_item, process_wallet_payment, add_reward_points	Encapsulate stock, wallet and rewards logic.
Triggers	Product update / delete audit	Automatically write audit information when product data changes.
Views	low_stock_view, daily_sales_view, expiry_products_view	Simplify repeated reporting and dashboard queries.

RELIABILITY

Transaction Safety

START TRANSACTION	FOR UPDATE	COMMIT	ROLLBACK
Begin atomic billing	Lock stock rows	Save successful bill	Undo failed bill

ENGINEERING CHOICES

A Few Design Decisions I'm Proud Of

Backend re-validation

The browser can validate quantities for convenience, but the backend recalculates and validates stock, prices and expiry before committing a bill.

Database-side errors

MySQL SIGNAL messages are mapped to clear application errors such as insufficient stock or insufficient wallet balance.

Hashed passwords

User passwords are checked with Werkzeug hashes rather than stored as plain text.

One-click demo launcher

A Windows BAT file starts the virtual-environment Python interpreter and opens the local application, which makes classroom demos quick and reliable.

SCOPE

What the Project Includes - and What It Does Not

Included	Intentionally out of scope

- ✓ Product / category / customer / supplier CRUD
- ✓ Inventory, low-stock and expiry management
- ✓ Billing with tax, discounts and multiple payment methods
- ✓ Wallet payments and reward points
- ✓ Authentication and password hashing
- ✓ Reports, audit logs and dashboard statistics
- ✓ Stored procedures, functions, triggers and views
- ✓ Transactions, rollback and row locking
- ✓ Dark mode and responsive UI
- ✓ BAT launcher for one-click local demos

- – Cloud deployment
- – Full production-grade role management
- – PDF invoice engine
- – Barcode / QR billing
- – Advanced refund workflow
- – Purchase-order workflow
- – Multi-store synchronization
- – Enterprise analytics and forecasting

REFLECTION

What I Learned

StoreLedger helped me move from writing isolated SQL queries to thinking about an application as a connected system. The most useful lessons were designing relationships before coding, keeping validation on the server, handling transactions carefully, and deciding which rules belong in Flask versus MySQL.

Database design

Relationships, constraints, views and normalization.

Stored programming

Procedures, functions, triggers and SIGNAL.

Transactions

Commit, rollback and row-level locking.

Backend development

Flask routes, sessions and server-side validation.

Frontend integration

Forms, tables, billing cart logic and responsive UI.

Project delivery

Git, documentation, README and a one-click demo workflow.

CONCLUSION

Final Thoughts

StoreLedger began as a DBMS mini-project, but it became a useful exercise in full-stack thinking. The project demonstrates how a browser interface, Flask backend and MySQL database can work together while still giving the database meaningful responsibility for calculations, validation, auditing and transaction safety.

For me, the strongest part of the project is not the number of screens. It is the connection between the screens and the data underneath them: a bill affects stock, wallet balances, rewards, orders and reports; product changes can be audited; and database views power application summaries. That end-to-end flow is what I wanted the project to show.

EXPLORE

Project Links

GitHub	https://github.com/tushar-011
LinkedIn	https://www.linkedin.com/in/tushar-thakur-8848a7396/
Email	artificial.thakur@gmail.com